End of Module Assignment

Module:

Advanced Object-Oriented Design and Programming

Course/Programme:

MSc Cyber Security

Word count:

2700

I. Task 1 Summary of Key Learning Outcomes

When I started the module my understanding of object oriented (OO) programming was mostly surface level. I knew what classes and objects were, but not how to use them to structure a real problem. Working through the exercises changed that. I moved from simply writing classes, methods, attributes and instances to thinking about why each one exists and what responsibility it should hold. Learning the SOLID principles created by Robert C. Martin gave me a vocabulary for that, and once I understood them, I started designing my code around them rather than writing it first and tidying up afterwards. By the end I could take a complex problem like a shared bank account and break it into clear, well separated classes, which is the real evidence that I met the objective of applying advanced OO principles.

Those same principles made me think differently about safety, because clean structure is only useful if the code also behaves correctly. In my BankAccount I added validation so that negative amounts are rejected and a penalty is applied, which stops invalid input from quietly corrupting the balance. These checks were not something I added at the end, they came from thinking about how the code could fail and designing against it, which is what secure coding really means to me now.

Once the structure was solid, design patterns were what tied everything together, and the idea that clicked most for me was the Open and Closed Principle, that code should be open to extension but closed to modification (Martin, 2023, ch. 4). It made sense once I saw it in my own work. When I wanted to add logging, I did not touch BankAccount at all, I just wrapped it in a LoggingAccount decorator. It was the same with transactions, where adding a new type means writing a new class instead of editing existing ones. That was the moment patterns stopped being theory and felt genuinely useful. In practice I saw three benefits clearly. I saw runtime flexibility through interchangeable Deposit and Withdraw classes that act as a Strategy or

Command (University of Essex Online 2026b). I saw separation of concerns through the decorator that adds logging without changing the account. And I saw extensibility through the Visitor pattern, where an outside algorithm can read an account without the account needing to change.

A small example is the accept method I added to BankAccount, which is all the account needs in order to let any visitor read it.

```
def accept(self, visitor):   # Visitor pattern (unit 5)  lets an outside algorithm read the account
    return visitor.visit_account(self)
```

Because the logic lives in the visitor and not the account, I can add new reports later without changing this class at all.

The hardest objective to meet was robustness, and it built directly on the safety work above, because a bank account is shared by many users at the same time. To handle this I gave each account its own lock, so only one thread can change the balance at a time and the total always stays correct. In the transfer method I went further and locked both accounts in a fixed order, which prevents deadlock when two transfers happen at once. Writing tests that ran a hundred threads together and still produced the exact expected balance showed me the design held up under pressure, which is the kind of reliability that large systems depend on.

Seeing how my small system had to cope with concurrency made me curious about how these same ideas work at a much bigger scale, which led me to some patterns used in AI and data science systems. The Model Registry is basically a central place to store machine learning models and keep track of their versions through their whole life, from training to production and eventually being retired. It saves things

like the version, metadata and where each model came from, so the right one can be deployed or rolled back without confusion. To me this felt like an extension of OO ideas I had already used, such as a registry or factory, one controlled place to handle models instead of spreading that logic everywhere. The Feature Store is similar in spirit. It is a central store for reusable features, so the same feature definitions can be shared across different models and stay consistent between training and serving. That lines up with OO principles like reuse and separation of concerns (DRY), just on a bigger, system wide scale.

Putting all this together, I started to see how the same habits make code ready for AI and data work. Because my classes have clean interfaces and clear responsibilities, a part can be swapped or extended without breaking the rest, which is exactly what you need when models or data pipelines change often. Anything that follows the same simple interface as BankAccount can be wrapped by the decorator or read by the visitor without being rewritten. This connects back to the Model Registry and Feature Store ideas, because clear boundaries and reuse are what let AI systems stay flexible, testable and maintainable as they grow.

II. Task 2 Showcase Artefacts: e-Portfolio: https://github.com/Elviix/E-portfolio

Module: https://github.com/Elviix/E-portfolio/tree/main/Advanced-Object-Oriented-Design-and-Programming

- The Artefact

My main artefact for this module is a thread safe banking program in Python. It contains a BankAccount class, a Transaction hierarchy with Deposit and Withdraw, a

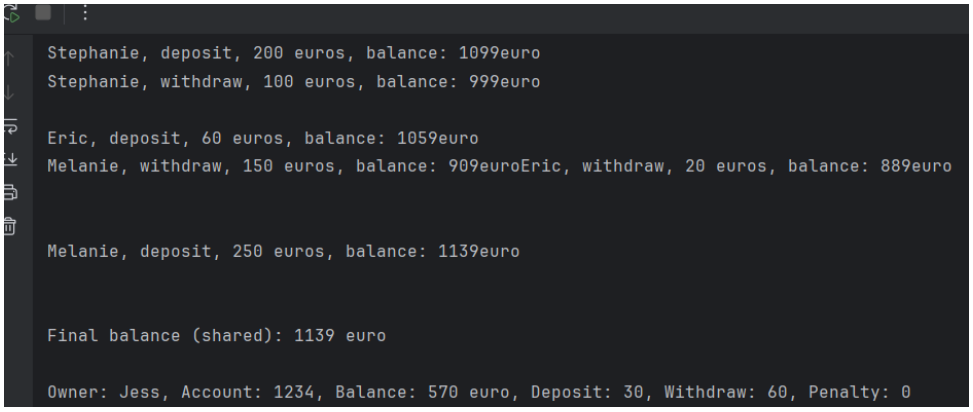
LoggingAccount decorator, an AccountVisitor with a ReportVisitor, a TransactionSimulator that runs several users at once, and a suite of twelve unit tests. I chose this single project on purpose, because it let me show coding, design patterns and testing all working together in one realistic system rather than as separate disconnected snippets.

- Object oriented principles used

The program relies on the core OO ideas throughout. Encapsulation keeps the balance and the lock private so they can only be changed through controlled methods. Abstraction and polymorphism appear in the abstract Transaction class, where every transaction is run the same way through execute() even though each one behaves differently. The SOLID principles guided the structure (Martin, 2023, ch. 3), especially single responsibility, since the account, the simulator and the tests each do one job, and the Open and Closed Principle, since new behaviour is added by writing new classes rather than editing existing ones.

- Coding exercise, thread safe code

The first part of the artefact is the concurrency work. Because the account is shared, I gave each account its own lock so only one thread can change the balance at a time, and in transfer I lock both accounts in a fixed order to prevent deadlock. This directly improves robustness, and I proved it with tests that run a hundred threads at once and still reach the exact expected balance.



Concurrent users running in parallel. The interleaved print ("909euroEric") shows genuine concurrency, while the shared balance stays exact.

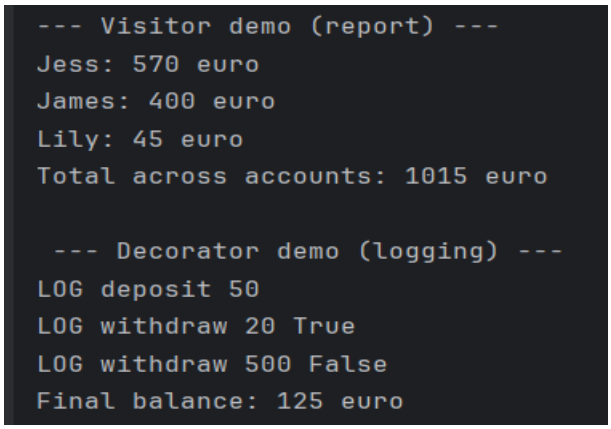
Final state of the accounts after the simulation, with deposits, withdrawals and penalties tracked per account.

- Design patterns

I chose the Command and Strategy idea for transactions (Martin, 2023, ch.4) (University of Essex Online 2026b) because it removes the need for a long chain of if statements on a transaction type. Each transaction is its own interchangeable object, which improves extensibility, since adding a new type is just a new class. I chose the Decorator pattern for LoggingAccount because I wanted to add an audit log without changing BankAccount, which improves maintainability by keeping the logging concern separate and also supports auditability. I chose the Visitor pattern for reporting because it lets an outside algorithm read accounts and total balances

without the account class needing to know anything about reports, which improves testability and keeps the account focused on its own job.

For completeness I also studied the Abstract Factory pattern (University of Essex Online 2026a), which creates families of related objects through one interface. I did not force it into this project because a single account type does not need it.

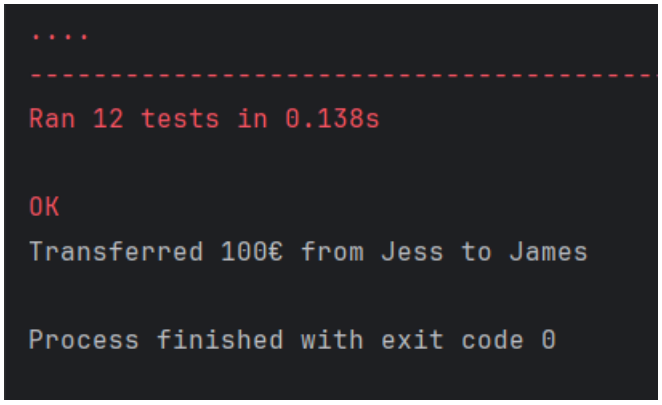


The Visitor totalling all accounts and the Decorator audit log, including a declined withdrawal of 500 euro.

- Testing, mocking and AI assisted testing

Testing was a major part of the artefact. As well as the behaviour and concurrency tests, I used a mock to test the decorator in isolation, a practice recommended in test driven development (Helmi, 2023). By passing a MagicMock in place of a real account I could check that LoggingAccount delegates correctly and returns the right result, without depending on the real BankAccount at all. This isolation is only possible because I used dependency injection, passing the account into LoggingAccount through its constructor, so any object with the same interface, real or fake, can be substituted without changing the decorator (University of Essex Online 2026c). This is the same technique that lets you test AI dependent code

without calling a live model or API, by replacing the slow or unpredictable part with a controlled stand in.



All twelve unit tests passing.

- Critical commentary, challenges and trade offs

The hardest challenge was concurrency. My early version lost updates when several threads changed the balance at once, and getting the locking right, especially the ordered locking in transfer, took several attempts before the totals stayed exact.

Watching the same test pass on one run and fail on the next was frustrating, but it forced me to stop guessing and to reason carefully about exactly where two threads could interleave. A second trade off was knowing when to stop adding patterns. The abstract base class is only justified because I have several transaction types and wrappers, so adding it for a single class would have been over engineering. Testing also brought its own complexity, since concurrent tests can pass or fail depending on timing, so I had to design them to give a definite expected result rather than relying on luck. Balancing clean abstraction against simplicity was a constant theme, and the project taught me that a pattern is only worth using when it earns its place.

Task 3 Reflection on Skill Development

- Critical thinking and analysis

The biggest academic skill this module built was the ability to look at a messy problem and break it down before writing any code. When I started the Python module I used to get the blank page syndrome, where I simply did not know how to begin. Learning OOP gave me structure and understanding, and that changed how I approach a problem. With the bank account I had to think about what could go wrong, shared access, bad input, two transfers at once, and then decide which OO principle answered each risk. Analysing the problem this way, instead of coding first and fixing later, is a habit I did not have at the start.

- Ethical considerations in AI driven OO design

The module also made me think about ethics in a way I had not connected to code before. Poorly designed class hierarchies can quietly carry bias forward, because if a flawed assumption is baked into a base class everything that inherits from it repeats the same flaw. AI enabled objects raise the further problem of transparency, since a model hidden inside a class can behave like a black box that no one can explain. I came to see that an OO designer has real responsibility here, to keep classes testable, to avoid black box coupling where one part secretly depends on another, and to support auditability through clean design. My LoggingAccount decorator is a small example of that last point, because an audit trail is exactly what lets someone check later what the system actually did.

- Problem solving

My problem solving improved most through refactoring. From the start I tried to write clean and reusable code, and for this assignment I actually reused the code I had written for the previous exercise in unit 6 and improved it. That experience showed me that good code is rarely finished in one go. I added validation, then a decorator, then a visitor, and each change had to fit without breaking what already worked. Being able to build on my earlier work instead of starting over proved the point of writing reusable code in the first place, and the tests gave me confidence that each step was safe.

- Research and self study

Working independently was a skill in itself. On top of the module support and the set books, I relied a lot on GeeksforGeeks, YouTube Python channels and Codacademy , as well as practitioner articles on test driven development (Helmi, 2023), to fill the gaps and see ideas explained in different ways. I used these to understand things like threading and locks properly rather than just copying examples, and to understand the reasoning behind the design patterns and not only their shape. This wider reading is what let me explain in my own words why each pattern was the right choice.

- Time management

Time management is the employability skill I still need to improve, especially around the readings. I managed the coding and the written tasks reasonably well, and reusing a project I already understood saved me time, but I tend to fall behind on the longer reading. Recognising this is the first step, and for future modules I want to plan the reading into my week instead of leaving it until I need it.

- Communication and literacy

Communication improved through the collaborative discussions and the written tasks. Explaining my refactoring to peers and reading their solutions forced me to put technical ideas into clear language, and writing this reflection made me justify my decisions rather than just describe them. Putting reasoning into words is harder than writing the code, and it is a skill I can see being just as important in a real job.

- Resilience and interpersonal skills

Finally the module tested my resilience, mostly through the concurrency bugs that did not behave the same way twice, which taught me to stay patient and methodical instead of guessing. The group discussions helped too. My peers gave me good and very instructive feedback on my work, although my own replies were a bit shy, because as someone on retraining I sometimes felt a little illegitimate replying to people with an engineering background. Over time I realised that asking questions and sharing ideas is exactly how engineers actually work, and that my fresh perspective is worth contributing rather than holding back.

Task 4 Professional Development Plan (PDP):

Looking back at the module through Kolb's experiential learning cycle (Kolb, 1984) helped me understand not just what I learned but how I learn. My concrete experience was clear: I understood concepts quickly when I read them, but I did not code immediately on the topic I had just studied. Reflecting on this, I can see it cost me time, because by the time I sat down to implement something I often had to go back and read the material again. In short, I read more than I coded, and that was a mistake. The abstract lesson I take from it is that for me, theory only becomes knowledge once it passes through my hands. My active experimentation is already in

place: I now code for at least fifteen minutes every day and build small projects to consolidate what I read, and I intend to code alongside the reading rather than after it.

My strengths are on the analysis side. I grasped the SOLID principles well and I can break a problem down before writing code, which shows in the structure of my artefact. My main area for improvement is design patterns, where I was sometimes confused between patterns that look similar in shape but differ in intent, such as Strategy and Command. The daily practice is aimed directly at this gap, because patterns stopped confusing me only when I had implemented them myself.

My career goal is to work in digital forensics and incident response (DFIR), and the module's three future themes map onto that goal more directly than I first expected. AI-aware architecture matters because modern forensic tools increasingly use machine learning for triage, anomaly detection and log classification, and the clean interfaces and abstraction I practised in this module are exactly what allow an investigator to integrate or replace an AI component in an analysis pipeline without rebuilding it. Advanced testing for intelligent systems matters because forensic results must be reproducible and defensible, and my mocking work showed me how to validate a pipeline against controlled data instead of live evidence or a live model. Ethical engineering is where the connection is strongest: an AI verdict that cannot be explained is worthless in court, and building my LoggingAccount decorator taught me that auditability has to be designed in from the start, not bolted on afterwards. The same lesson applies to chain of custody.

Automation ties all of this together. A large part of DFIR work is Python automation, parsing logs, triaging artefacts and scripting repetitive analysis, and the object

oriented skills from this module are what will make those scripts maintainable tools rather than throwaway code. My concrete next actions are therefore: to keep the daily coding habit but pair it with the reading; to build a small applied project, a log parser that uses the Strategy pattern to handle different log formats, which practises both my weak area and my target field at once; and to work towards a recognised digital forensics certification such as the GIAC Certified Forensic Analyst (GCFA), supported by further reading on forensic tool validation.

References:

Helmi, A. (2023) 'Test-driven development in Python: best practices and detailed explanation', *DEV Community*, 29 June. Available at: https://dev.to/00geekinside00/test-driven-development-in-python-best-practices-and-detailed-explanation-42e8 (Accessed: 16 July 2026).

Kolb, D.A. (1984) *Experiential learning: experience as the source of learning and development*. Englewood Cliffs, NJ: Prentice-Hall.

Martin, R.C. (2023) *Functional design: principles, patterns, and practices*. Boston: Addison-Wesley.

University of Essex Online (2026)a 'Unit 3: Creational design patterns' [Lecturecast]. *Advanced Object-Oriented Design and Programming*, April. Available at: https://www.my-course.co.uk (Accessed: 16 July 2026).

University of Essex Online (2026)b 'Unit 5: Behavioural design patterns' [Lecturecast]. *Advanced Object-Oriented Design and Programming*, April. Available at: https://www.my-course.co.uk (Accessed: 16 July 2026).

University of Essex Online (2026)c 'Unit 11: Introduction to Dependency Injection (DI) and Inversion of Control (IoC)' [Lecturecast]. *Advanced Object-Oriented Design and Programming*, April. Available at: https://www.my-course.co.uk (Accessed: 16 July 2026).